

01. Storage

DBMS stores:

- Relations

Relation schemas Structure of relations, constraints, triggers

View Definitions

Indexes Derived information to speed up access to relations

Statistical information about relations for use by query optimizer

- Log files - Data recovery

Primary memory Registers, static RAM (cache), dynamic RAM (physical mem)

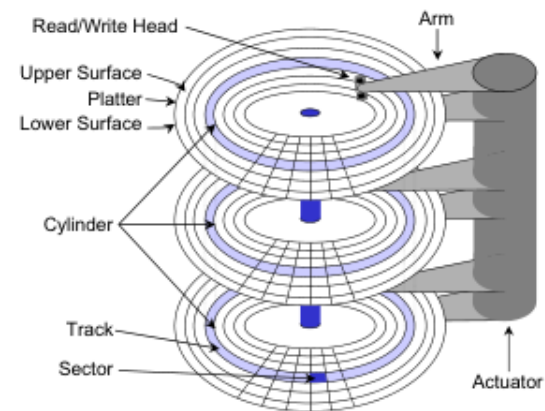
Secondary memory Magnetic disk (HDD), solid-state disks

Tertiary memory Optical disks, tapes

Tradeoff: Capacity, cost, access speed, volatility

- DBMS stores data on non-volatile disk for persistence

- DBMS processes data in main memory



Disk access time **Command processing time** Interpreting access command by disk controller

Seek time Moving arms to position disk head on track 5-6ms

Rotational delay Waiting for block to rotate under head (half revolution)

Transfer time Actually moving data to/from disk surface (100-200microsecond)

Access time = seek time + rotational delay + transfer time

SSD

- NAND flash memory without moving parts
- Random: 100x Faster, Sequential: Slightly faster
- Lower power consumption
- Disadvantages:

- Update to a page requires erasure of multiple pages before overwriting page
- There is a limited number of times a page can be erased

Storage Manager Components

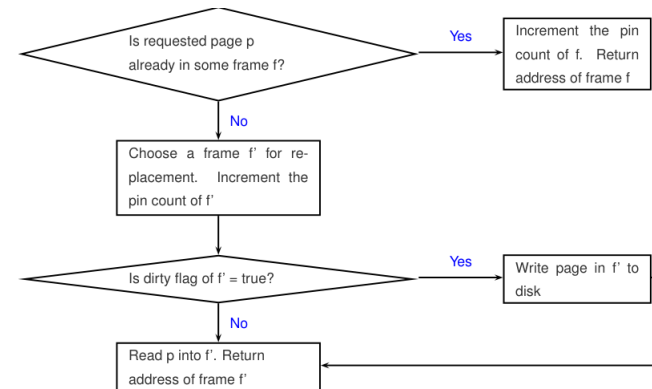
- File & access methods Manager
- Buffer pool manager

– Controls reading/writing of disk pages

- Disk space manager

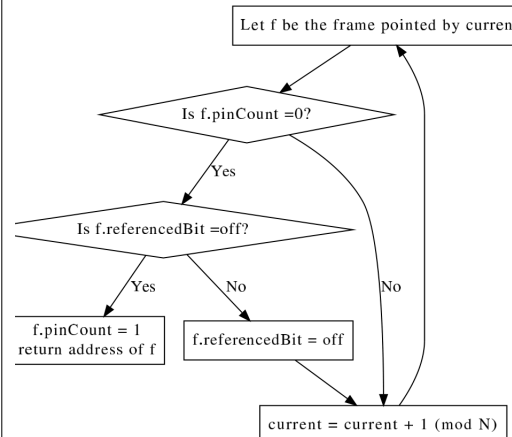
Buffer Manager

- *Buffer pool* = main memory allocated for DBMS
- Partitioned into block-sized pages called frames



Replacement policy

- Random
- First in First Out
- Most Recently Used
- Least Recently Used
- Clock - current variable, referenced bit (replace page with referenced bit and pin out = 0)



Files

File abstraction: Operations

- Create, delete, insert/delete/get records w RFID, scan

File Organisation = Arranging data records in a file that is stored on disk

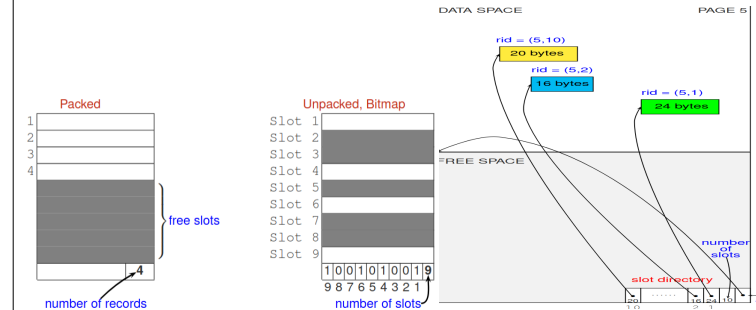
- Heap File: Unordered files

– Linked List Implementation: Header Data pages

– Page Directory Implementation w Fixed length records: RID = (page id, slot number)

- Sorted File: Records ordered on search key
- Hashed file: Records in blocks via hash functions

Page Organisation



- Slotted page organisation splits pages into data vs free space
- RID references the position of pointer in the slot directory
- Maintaining RID means that the position of the pointer must be the same

Forwarding record RID is unchanged where record is another pointer to the location of the data

Lecture 2: B+ Tree Index

Index Data structure to speed up retrieval of data records

Search key Sequence of $k \geq 1$ data attributes

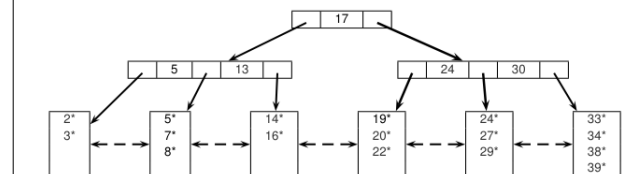
- Composite search key if $k > 1$

Unique index Search key contains a candidate key of index table

Tree-based index Based on sorting of search key values eg. ISAM, B+

Hash-based index Data entries accessed using hashing eg. static, extendible, linear

B+ Tree



Internal nodes store index entries of the form $(p_0, k_1, p_1, k_2, p_2, \dots, p_n)$

- ▶ $k_1 < k_2 < \dots < k_n$
- ▶ p_i = disk page address (root node of an index subtree T_i)
- ▶ For each data entry k^* in T_0 , $k < k_1$
- ▶ For each data entry k^* in T_i ($i \in [1, n)$), $k \in [k_i, k_{i+1})$
- ▶ For each data entry k^* in T_n , $k \geq k_n$

Each (k_i, p_i) is an **index entry**; k_i serves as a **separator** between the node contents pointed to by p_{i-1} & p_i

Order of index tree Controls space utilization

- Non-root nodes contains m entries, where $m \in [d, 2d]$
- Root node contains m entries, where $m \in [1, 2d]$

Equality Search

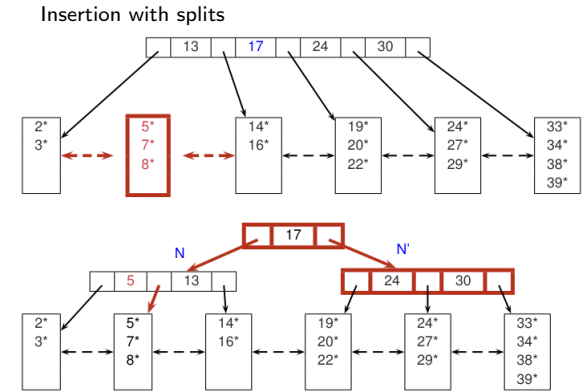
- At each internal node N , find largest key k_i in N such that $k_i \leq k$
- If k_i exists, search subtree at p_i

Formats of data entries

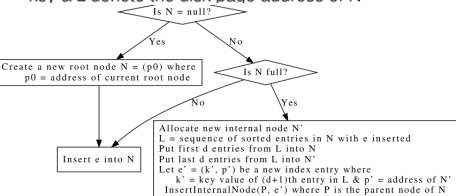
Format 1 Actual data record

Format 2 (k, rid) where rid is record identifier of data record

Format 3 (k, rid-list), format 2 but can point to multiple data records

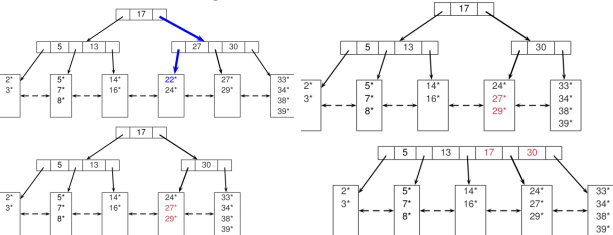


- Split the **overflowed internal node N** by distributing d entries to a **new right adjacent node N'** & inserting (M, ■) into its parent node, where M is the middle key & ■ denote the disk page address of N'

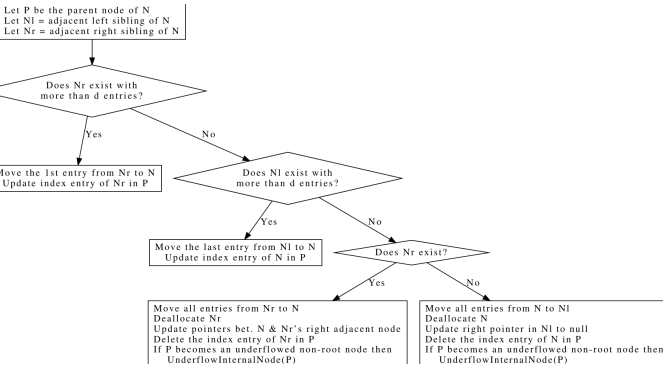


- Split key goes to the right node (distribute d+1 entries to new leaf)
- Redistribution only checks right sibling then left sibling else node split

Deletion with merges



- Redistribute by taking from adjacent sibling node first
- Should only merge where adjacent sibling node has less than m entries
- Redistribution of internal entries attempted whenever possible



Clustered index Data is stored together with its RID

Unclustered index Relation is stored in a heap file referenced by this index

Bulk loading of B+ Tree

- Sort the indices and place them in 2*d buckets
- Take first number of each bucket as the separator of internal node
- Propagate internal node and parent nodes once the internal node has 2*d entries

Lecture 3: Hash-based Indexing